Reloading Inventory, Recipe, Batch, and Storage Tracking Application

Updated Requirements

Revision source: `Reloading App_Specifications.pdf` plus the current repository implementation.

Revision date: 2026-06-19

Revision Summary

This version updates the original requirements document to match the implemented application now present in the repository.

Added

- Audited inventory adjustment workflow, including manual corrections, "deplete remaining", and restoration of depleted lots.
- Explicit account reset CLI and local reset workflow.
- UUID public/user-facing identifiers for recipes and batches.
- Friendly two-word batch slugs retained separately from batch identifiers.
- Suggested two-word recipe titles.
- Container cartridge capacity and capacity enforcement.
- Derived batch storage and depletion states driven by container assignment and container use.
- Tracking of batch quantity cleared from emptied containers.
- Garmin Xero C1 Pro FIT import into batch performance records.
- Settings page backup workflow.
- Tenant-scoped JSON and CSV exports.
- Selenium browser workflow tests, including a complete .357 Magnum workflow.
- Docker Selenium profile for opt-in end-to-end browser testing.
- Help/context download endpoint for LLM-oriented app context.

Removed or Deferred

- Recipe component alternatives are no longer supported in the implemented workflow. Each core recipe role may have only one exact item. To use a different primer, case, bullet, or powder, the user creates a separate recipe.
- Recipe two-word slugs were replaced by UUID identifiers. Two-word generation is now used for suggested recipe titles, not recipe IDs.
- Direct binary upload of source materials is not implemented. Source records can store citation, URL, page, file name metadata, and notes.
- Return-to-new-lot behavior is not implemented as part of the return endpoint. Returned inventory can be credited to the source lot or to an

existing compatible lot; a new lot can be created separately through the inventory workflow.
- Google OAuth, email-based password recovery, nested containers, full container occupancy history, and load-data recommendation remain future candidates.

Modified

- Batch lifecycle now separates production completion from storage assignment. A batch moves from `UNDER PRODUCTION` to `PRODUCED`; storage-related states are derived from container assignments.
- Batch depletion state names now use `PARTIALLY DEPLETED` and `DEPLETED` instead of `PARTIALLY USED` and `USED`.
- `PARTIALLY USED` and `USED` are container states, not batch states.
- Container `RETIRED` is not implemented in the current state machine.
- Inventory lot `opened_on` is system-managed and set when an active lot first has drawdown, not accepted from lot creation input.
- Active inventory lots can be replaced during lot creation when the user explicitly requests replacement.
- Performance/quality records cannot be created while a batch is still `UNDER PRODUCTION`.
- Public recipe views expose only public-safe recipe information and `public_notes`; private notes, source notes, inventory, batches, containers, tokens, and user details remain private.

1. Purpose

The application provides a multi-user, multi-tenant web system for tracking ammunition reloading components, inventory lots, user-defined cartridge recipes, production batches, storage containers, QR labels, performance/quality results, backups, exports, and audit history.

The primary goal remains traceability. The application shall allow a user to determine:

- What component items were defined.
- What inventory lots were acquired.
- Which inventory lots were active, reserved, consumed, adjusted, depleted, returned, or lost.
- Which recipe a batch was produced from.
- Which exact lots were reserved and consumed by a batch.
- Which containers currently hold completed cartridges from a batch.
- Which quantities have been cleared from emptied containers.
- What performance and quality information was recorded for each batch.
- How batch performance contributes to recipe-level evaluation.
- Which safety or traceability warnings were acknowledged.
- Which important operations occurred in audit history.

The application stores user-entered load data. It does not recommend powder charges, infer safe loads, certify that a recipe is safe, or replace published manuals or manufacturer data.

The architecture and data model shall remain open to future analytics, source comparison, and import workflows, but recommendation-like behavior shall not be part of the current implementation.

2. Project Environment

2.1 Runtime

The application uses:

- Python 3.12.
- Docker Compose based runtime.
- Pip-managed dependencies from `requirements.txt`.
- SQLite as the current database engine.
- Pytest for unit, functional, and workflow testing.
- Selenium for opt-in browser workflow testing.

2.2 Deployment

The application shall run as a fresh system through Docker Compose.

The current Compose stack includes:

- `storage`: Flask JSON API, SQLAlchemy model, business rules, audit records, Alembic migrations, and SQLite ownership.
- `renderer`: separate Flask/Jinja browser app that calls the storage API and does not directly open the database.
- `web`: Nginx browser-facing static and reverse proxy entrypoint.
- `selenium`: optional Selenium standalone Chrome service under the `selenium` Compose profile.

The storage container runs pending Alembic migrations before starting. The SQLite database is stored in the `reloading-data` Docker volume at `/data/reloading.sqlite3`.

2.3 Libraries

The implemented application uses:

- Flask.
- Flask-SQLAlchemy.
- Flask-Migrate.
- SQLAlchemy.
- Alembic.
- Jinja2 through Flask templates.
- Gunicorn.

- Requests.
- Werkzeug password hashing.
- qrcode and Pillow.
- Pytest and pytest-cov.
- Selenium.

The renderer uses server-rendered pages plus static JavaScript for page-specific interactions.

3. Application Architecture

3.1 Storage Service

The storage service is the backend system of record. It is responsible for:

- Database access.
- Data validation.
- Business rules.
- Authentication and bearer-token session validation.
- Tenant ownership enforcement.
- Inventory reservation, consumption, return, loss, adjustment, and depletion logic.
- Recipe, batch, container, and performance lifecycle transitions.
- Audit logging and acknowledgement storage.
- REST-style JSON API endpoints.
- QR code image generation.
- JSON and CSV export generation.
- SQLite backup creation.
- Alembic migration execution on container startup.

The storage service owns the SQLite database.

3.2 Rendering Application

The rendering application is a separate Flask app responsible for:

- Rendering browser pages.
- Presenting forms and workflows.
- Calling storage APIs over HTTP.
- Managing browser sessions for the rendered UI.
- Showing validation errors, warnings, acknowledgements, and dashboard metrics.
- Serving download links for QR codes, exports, backups, and help/context text.

The rendering application shall not directly manipulate the database.

3.3 Static HTTP Point of Contact

Nginx is the browser-facing entrypoint. It is responsible for:

- Serving static assets from the rendering app.
- Routing browser requests to the renderer.
- Providing a future TLS termination point.

4. Multi-Tenant User Model

4.1 Tenancy

The application is multi-tenant. Each user has a separate dataset.

A user shall not be able to view, modify, infer, or access another user's items, inventory lots, recipes, batches, containers, performance records, audit records, acknowledgements, or exports.

All tenant-scoped entities include a `user_id` ownership relationship. All write operations and all private read operations enforce ownership.

4.2 Authentication

The current implementation supports username/password authentication where the username is the user's email address.

Passwords are securely hashed with Werkzeug.

Registration requires:

- Email address containing `@`.
- Password with at least 10 characters.

Google OAuth remains a future candidate and is not implemented.

4.3 Sessions

The storage API issues bearer tokens. Tokens are stored server-side as SHA-256 hashes in `auth_session`.

Sessions include:

- Owning user.
- Token hash.
- Created timestamp.
- Expiration timestamp.
- Optional revoked timestamp.

Session lifetime is configured by `SESSION_HOURS`, defaulting to 12 hours.

4.4 Password Reset

Email-based password recovery is not implemented.

The current reset workflow is local and operator-initiated:

- An operator runs `flask --app storage_service.app mark-reset user@example.com`.
- Existing active sessions for that user are revoked.
- Login returns a `password_reset_required` response.
- The rendering app directs the user to `/reset-password`.
- The user enters email and a new password.
- Reset is accepted only if `LOCAL_RESET_ENABLED=true` and the account is marked `reset_required`.

4.5 Roles and Sharing

The implemented sharing model is limited to recipe public links.

Recipes support:

- Private.
- Public via public token link.

Public recipe access is view-only and does not expose private inventory, batches, containers, private notes, source notes, public token, user id, or unrelated recipes.

No user-to-user viewer role assignment is implemented.

5. Core Domain Concepts

The implemented domain entities are:

- User.
- Auth Session.
- Item.
- Inventory Lot.
- Inventory Adjustment.
- Recipe.
- Recipe Component.
- Source Material.
- Batch.
- Batch Inventory Reservation.
- Batch Inventory Consumption.
- Inventory Return.
- Storage Container.
- Container Assignment.
- Performance/Quality Record.
- Audit Log.

- User Acknowledgement.

6. Item Requirements

6.1 Definition

An Item represents a reusable catalog definition of a reloading component or related supply.

An Item is not itself inventory.

Examples:

- Hornady XTP .357 158 grain JHP bullet.
- Winchester 296 powder.
- CCI 550 small pistol magnum primer.
- Starline .357 Magnum brass.
- MTM adhesive cartridge labels.

6.2 Item Categories

Implemented categories are:

- `BULLET`.
- `POWDER`.
- `PRIMER`.
- `CASE`.
- `COMPLETED CARTRIDGE`.
- `OTHER`.

Unknown submitted categories are normalized to `OTHER`.

6.3 Item Attributes

Items track:

- Owning user.
- Category.
- Manufacturer.
- Product line.
- Name.
- Differentiating characteristics.
- Caliber, where category-specific.
- Bullet weight, where category-specific.
- Bullet type, where category-specific.
- Primer type, where category-specific.
- Powder type, where category-specific.
- Flexible JSON attributes.
- Notes.
- Archived flag.

- Created timestamp.
- Updated timestamp.

Category-specific fields are accepted only for compatible categories. Fields submitted for other categories are ignored instead of being stored incorrectly.

Items can be archived and omitted from default lists. They are not normally hard-deleted.

7. Inventory Lot Requirements

7.1 Definition

An Inventory Lot represents a specific acquired quantity of an Item.

Inventory is lot-based. Lots are user-defined and commonly map to purchase or acquisition events rather than every physical package.

7.2 Lot Traceability

Inventory lots track:

- Owning user.
- Referenced item.
- Manufacturer lot number.
- Date acquired.
- System-managed date opened.
- Original quantity.
- Original unit.
- Backend-normalized quantity.
- Backend-normalized unit.
- Optional total acquisition cost.
- Adjustment quantity.
- Available quantity.
- Reserved quantity.
- Consumed quantity.
- Depleted state.
- Active consumption state.
- Notes.
- Created timestamp.
- Updated timestamp.

Available quantity is derived as:

`normalized_quantity + adjustment_quantity - reserved_quantity - consumed_quantity`.

7.3 Active Lot Rule

For a given user and item, only one non-depleted lot may be active at a time.

The user can create additional inactive lots. A new active lot can explicitly replace the currently active lot when `replace_active` is requested. The replaced lot is deactivated and audited.

An active depleted lot is automatically deactivated.

7.4 Opened Date

`opened_on` is system-managed.

The lot is marked opened when it is active and first has inventory drawdown through reservation or consumption. A creation payload cannot set `opened_on`.

If a lot had prior drawdown while inactive and the user later activates it, the system sets `opened_on` at activation.

7.5 Historical Lots

Depleted lots remain visible for historical purposes.

Default inventory lists hide depleted lots. Historical/depleted lots can be shown through a filter.

Historical lots shall not be hard-deleted through normal workflows.

7.6 Unit Handling

The user selects a unit when adding inventory.

Powder units are normalized to grains. Supported powder units include:

- Grain/grains/gr.
- Ounce/ounces/oz.
- Pound/pounds/lb/lbs.
- Gram/grams/g.
- Kilogram/kg.

Count-based categories normalize to count and require whole-number quantities.

Count units include:

- Count.
- Each.
- Ea.
- Piece/pieces.

7.7 Inventory Adjustments

The application supports audited inventory adjustments for an inventory lot.

An adjustment records:

- Owning user.

- Inventory lot.
- Created timestamp.
- Quantity change.
- Unit.
- Available quantity before.
- Available quantity after.
- Required reason.
- Notes.

Rules:

- Adjustment quantity must be non-zero.
- Count-based adjustments must be whole numbers.
- Adjustment cannot reduce available quantity below zero.
- Adjustment is blocked while the lot has active reservations.
- The user can request `deplete_remaining`, which creates an adjustment equal to negative available quantity.
- If the adjustment depletes the lot, the lot is marked depleted and inactive.
- A positive later adjustment can restore availability and clear depleted state, but it does not automatically reactivate the lot.

8. Recipe Requirements

8.1 Definition

A Recipe represents one completed cartridge.

A Recipe is user-defined. It references exact Items, not broad item categories. It does not consume inventory directly.

Inventory is reserved and consumed only when a Batch is created and transitioned through production completion.

8.2 Recipe Identity

Each recipe has:

- Internal integer database id.
- User-facing UUID identifier returned as `id`.
- User-entered title.

The previous two-word recipe slug requirement has been replaced. The application now provides a suggested two-word title through `/api/recipes/suggested-identity`; the recipe identifier itself is UUID-based.

Suggested two-word titles are generated from the same verb/noun word lists used for friendly slugs and are unique within the user's recipe titles at generation time.

8.3 Recipe Lifecycle

Recipe states are:

- UNDER DEVELOPMENT.
- UNDER TEST.
- APPROVED.
- NOT APPROVED.
- RETIRED.

Allowed transitions are:

- UNDER DEVELOPMENT -> UNDER TEST.
- UNDER TEST -> UNDER DEVELOPMENT.
- UNDER TEST -> APPROVED.
- UNDER TEST -> NOT APPROVED.
- APPROVED -> UNDER TEST.
- APPROVED -> RETIRED.
- RETIRED -> UNDER DEVELOPMENT.

Moving to UNDER TEST or APPROVED requires required recipe data. Missing source material may be overridden only with acknowledgement. Missing core cartridge/components block advancement.

NOT APPROVED is terminal.

8.4 Recipe Components

A recipe defines the components and quantities required to produce one completed cartridge.

Core roles are derived from the selected item's category. The API ignores a submitted role value and uses the item category.

Implemented core component roles:

- Bullet item, quantity in count.
- Powder item, charge in grains.
- Primer item, quantity in count.
- Case item, quantity in count.

Rules:

- A recipe may contain only one component for each core role.
- Component quantity must be positive.
- Powder recipe components require grain units.
- Count-based recipe components require count units.
- Components cannot be removed after a batch references the recipe.

Recipe component alternatives are disabled. The `alternative_group` field is preserved in the schema for legacy compatibility but is set to null by migration and not exposed in component JSON.

8.5 Recipe Parameters

Recipes support:

- Title.
- Cartridge/caliber.
- Overall length.
- Case length.
- Crimp type.
- Seating depth.
- Source/reference notes.
- Private notes.
- Public notes.
- Public/private sharing state.
- Archived flag.
- Created timestamp.
- Updated timestamp.

Firearm-used data belongs to performance/quality records, not recipe fields.

8.6 Source Material

Recipes support one or more source material records.

Source material records support:

- Kind.
- Citation.
- URL.
- Page.
- File name metadata.
- Notes.

At least one of citation, URL, file name, or notes is required.

Binary file upload is not implemented.

8.7 Recipe Safety Boundary

The application may:

- Store user-entered recipe data.
- Require completeness of certain fields.
- Warn when required fields are missing.
- Track source references.
- Track whether the user acknowledged responsibility or missing source material.
- Aggregate user-entered performance data from batches.

The application shall not:

- Infer safe loads.
- Generate powder charge recommendations.
- Claim a recipe is safe.
- Claim a recipe is suitable for a firearm.
- Replace published reloading manuals or manufacturer data.

8.8 Recipe Public Sharing

A recipe may be marked public. If public, the system generates a public token.

Public recipe JSON and pages expose:

- Public-safe recipe fields.
- Exact component item descriptions.
- Source material records.
- Public notes.

Public recipe output does not expose:

- Private inventory lots.
- Batch data.
- Storage containers.
- Private notes.
- Source notes.
- Public token.
- User id.
- Other recipes.

Archived recipes are not available through public tokens.

9. Batch Requirements

9.1 Definition

A Batch represents a production run based on one Recipe.

A Batch consists of one or more iterations of the recipe. For example, a batch of 100 cartridges represents 100 recipe iterations.

A Batch reserves and later consumes specific Inventory Lots.

9.2 Batch Identity

Each batch has:

- Internal integer database id.
- User-facing globally unique UUID identifier returned as `id`.
- Friendly two-word slug unique within the owning user's dataset.

The UUID identifier is used in API routes and QR URLs. The slug remains useful for display.

9.3 Batch Lifecycle

Implemented batch states are:

- `UNDER PRODUCTION`.
- `PRODUCED`.
- `PARTIALLY IN STORAGE`.
- `IN STORAGE`.
- `PARTIALLY DEPLETED`.
- `DEPLETED`.
- `CANCELLED`.
- `DECOMMISSIONED`.

Allowed manual transitions are:

- `UNDER PRODUCTION` -> `PRODUCED`.
- `UNDER PRODUCTION` -> `CANCELLED`.
- `PRODUCED` -> `DECOMMISSIONED`.
- `PARTIALLY IN STORAGE` -> `DECOMMISSIONED`.
- `IN STORAGE` -> `DECOMMISSIONED`.
- `PARTIALLY DEPLETED` -> `DECOMMISSIONED`.

`DEPLETED`, `CANCELLED`, and `DECOMMISSIONED` are terminal states.

Storage and depletion states are usually derived automatically from container assignments and container state changes:

- No assigned or depleted container quantity after production: `PRODUCED`.
- Some but not all quantity assigned to non-depleted containers: `PARTIALLY IN STORAGE`.
- Full quantity assigned to non-depleted containers: `IN STORAGE`.
- Some assigned or cleared quantity is associated with partially used, used, or emptied containers: `PARTIALLY DEPLETED`.
- All produced quantity is assigned/cleared and all assigned containers are used or empty: `DEPLETED`.

9.4 Batch Creation

When creating a batch, the user shall select:

- Recipe.
- Number of recipe iterations.
- Exact inventory lot allocations for every recipe component.
- Notes.
- Required acknowledgements where applicable.

The system calculates required inventory from recipe component quantities multiplied by iterations.

Every recipe component must be fully allocated. The sum of allocations for each component must exactly equal the required quantity.

Creating a batch reserves inventory immediately and places the batch in `UNDER PRODUCTION`.

The batch exposes a derived material-cost summary. While under production, cost is based on outstanding reserved inventory plus any production loss already consumed. After production completion, cost is based on committed consumption plus production loss. Cost per cartridge is the derived material cost divided by batch iterations. If any traced lot lacks cost, the cost-per-cartridge metric is unavailable rather than partially calculated.

9.5 Inventory Reservation and Consumption

The application uses a two-step inventory model:

1. Batch creation reserves inventory while the batch is `UNDER PRODUCTION`.
2. Transitioning the batch to `PRODUCED` commits all reserved inventory as consumed.

Reservation effects:

- Increase lot `reserved_quantity`.
- Reduce derived available quantity.
- Create `BatchInventoryReservation` rows with status `RESERVED`.
- Audit inventory reservation.
- Mark an active lot opened if this is its first drawdown.

Production completion effects:

- Decrease lot `reserved_quantity`.
- Increase lot `consumed_quantity`.
- Create `BatchInventoryConsumption` rows.
- Mark reservations `CONSUMED`.
- Skip reservations already marked `REPLACED` by production loss accounting.
- Mark depleted lots depleted and inactive.
- Audit inventory consumption.

Performance/quality data cannot be recorded while the batch remains `UNDER PRODUCTION`.

9.6 Inventory Shortage Handling

The user shall not be allowed to override inventory limitations.

Rules:

- Selected lots must belong to the user.
- Selected lots must match the component item.
- Selected lots must not be depleted.
- Selected lot available quantity must be sufficient for the allocation.
- If shortage or allocation mismatch occurs, batch creation fails and reservations are rolled back.

9.7 Multi-Lot Consumption

Multi-lot consumption is supported through explicit allocation rows.

Each allocation includes:

- Recipe component id.
- Inventory lot id.
- Quantity.

The system preserves traceability to every reserved and consumed lot.

When active-lot consumption depletes the current active lot and exactly one inactive consumed successor lot exists for the same item, the successor lot is automatically promoted to active and marked opened.

9.8 Production Loss and Replacement Accounting

Production loss is used while a batch is `UNDER PRODUCTION` to account for reserved material that was lost during the loading process and must be replaced before the finished batch can still satisfy the recipe quantity.

When recording production loss, the user defines:

- Source outstanding reservation.
- Quantity lost.
- Replacement inventory lot, unless the original lot has enough unreserved available inventory to replace the loss.
- Reason.
- Notes.

Rules:

- The unit is derived from the selected reservation lot; the user does not enter a unit.
- Production loss quantity must be positive.
- Count-based production loss must be a whole number.
- Production loss cannot exceed the selected outstanding reservation.
- Replacement lot must belong to the same user and contain the same item as the selected reservation.
- Replacement lot must have enough available inventory for the lost quantity.

Production loss effects:

- Decrease the source reservation by the lost quantity, or mark it `REPLACED` if the full reservation was lost.
- Increase the source lot `consumed_quantity` by the lost quantity.
- Create a new replacement reservation for the same batch component.
- Increase the replacement lot `reserved_quantity`.

- Create an auditable `BatchProductionLoss` record linking the source reservation, source lot, replacement reservation, and replacement lot.
- If the source lot has no remaining reserved or available inventory, mark it depleted/inactive and promote the selected replacement lot when no other active compatible lot exists.

9.9 Cancellation and Return/Loss Accounting

A batch in `UNDER PRODUCTION` can be cancelled only after every outstanding reserved quantity has been explicitly accounted for through the return/loss workflow.

The system shall not automatically assume all reserved components were returned.

For each lot with outstanding reservation, the user must enter returned plus lost quantity equal to the outstanding reserved quantity for that lot.

9.10 Batch Performance/Quality Data

Each batch may have one consolidated performance/quality record.

The record can be created after production completion. Edits set an `edited` flag and are audited.

10. Inventory Return Requirements

10.1 Definition

Inventory Return is the workflow used to account for returned, recovered, discarded, lost, or corrected components after a batch has reserved or consumed inventory.

Inventory return is used for:

- Cancelled batches.
- Decommission or correction workflows.
- Disassembled rounds.
- Partial recovery.
- Cancellation, decommission, and recovery loss accounting.

Production loss during an `UNDER PRODUCTION` batch is handled by the production loss workflow, not by inventory return.

10.2 Return Behavior

When performing an inventory return, the user explicitly defines:

- Batch involved.
- Source inventory lot.
- Quantity returned.

- Quantity lost or unrecoverable.
- Destination inventory lot, when returning to a lot other than the source.
- Reason.
- Notes.

Rules:

- Returned and lost quantities cannot be negative.
- Returned plus lost must be positive.
- Destination lot, if supplied, must belong to the same user and same item.
- For outstanding reservations, returned plus lost must exactly equal outstanding reserved quantity for that source lot.
- For already consumed inventory, return quantity cannot exceed consumed quantity for that batch and lot.
- Lost quantity remains consumed.
- Return-to-new-lot is not part of the implemented return endpoint.

Every return/loss operation creates a user acknowledgement of type `INVENTORY_RETURN_LOSS`.

10.3 Return Auditability

All inventory return operations are auditable.

The audit record includes:

- User.
- Timestamp.
- Batch identifier.
- Source lot.
- Destination lot, if any.
- Returned quantity.
- Lost quantity.
- Reason.
- Notes where applicable.

11. Storage Container Requirements

11.1 Definition

A Storage Container represents a physical container that can hold completed cartridges from one or more batches.

The current implementation does not support nested containers or location hierarchy.

11.2 Container Attributes

Storage containers track:

- Owning user.
- Container identifier, unique per user.
- Name or label.
- Cartridge limit.
- Description.
- Current state.
- Notes.
- Created timestamp.
- Updated timestamp.

Implemented container states are:

- `EMPTY`.
- `ASSIGNED`.
- `PARTIALLY USED`.
- `USED`.

`RETIRED` is not currently implemented.

11.3 Container Capacity

Every container requires a positive whole-number cartridge limit.

Assignments cannot exceed:

- Remaining capacity of the container.
- Remaining unassigned/non-depleted quantity of the batch.

Container capacity cannot be edited below the currently assigned quantity.

11.4 Batch Assignment

A batch can be split across one or more containers.

A container can hold one or more batches.

Rules:

- Batch must be produced before assignment.
- `UNDER PRODUCTION`, `CANCELLED`, `DECOMMISSIONED`, and `DEPLETED` batches cannot receive new container assignments.
- A `USED` container must be transitioned to `EMPTY` before new assignment.
- If a container already contains a different batch, the user must acknowledge a mixed-batch container.
- Reassigning more quantity of the same batch to the same container increments the existing assignment.
- Container assignment updates the batch storage state.

11.5 Container Quantity Tracking

The system shows:

- Container total live quantity.
- Remaining capacity.
- Quantity per batch.
- Recipe associated with each assignment.
- Batch identifier and slug.
- Batch state.
- Container state.

When a container transitions to `EMPTY`, assignments are cleared and each affected batch's `container_depleted_quantity` is incremented by the cleared quantity.

11.6 Container History

The current implementation tracks current assignments plus the aggregate batch quantity cleared from emptied containers.

It does not keep a full historical occupancy ledger for every container. Full container history remains a future expansion candidate.

11.7 QR Codes and Labels

The application generates QR codes for recipes and batches.

QR behavior:

- Batch QR codes point to authenticated batch pages.
- Private recipe QR codes point to authenticated recipe pages.
- Public recipe QR codes point to public recipe pages when the recipe is public and has a public token.

QR images are downloadable through the renderer.

12. Performance and Quality Requirements

12.1 Definition

Performance and quality data is stored separately from Recipes and Batches as a dedicated record type.

Each Batch may have one consolidated Performance/Quality Record.

Recipe-level performance and quality views are derived from related batch records.

12.2 Batch Performance/Quality Record

The record supports:

- Date recorded.
- Firearm used.
- Barrel length.

- Distance.
- Group size.
- Shot count.
- Velocity average.
- Velocity minimum.
- Velocity maximum.
- Standard deviation.
- Extreme spread.
- Temperature.
- Weather notes.
- Reliability notes.
- Pressure sign notes.
- Recoil perception.
- Accuracy perception.
- Cleanliness perception.
- Subjective rating.
- General notes.
- Raw data.
- Processed JSON data.
- Created timestamp.
- Updated timestamp.
- Edited-from-original indicator.

12.3 Garmin Xero C1 Pro Data

The application supports manual entry of chronograph-compatible fields and raw data.

The application supports importing Garmin Xero C1 Pro FIT files into a batch performance record. Imported chronograph values populate the recorded date, shot count, velocity average, minimum, maximum, standard deviation, extreme spread, raw data, and processed JSON fields.

After a Garmin import, programmatically imported chronograph fields are displayed read-only. User-entered contextual fields such as firearm, barrel length, distance, group size, temperature, perception ratings, subjective rating, and general notes remain editable.

12.4 Recipe Aggregation

Recipe detail responses include aggregate performance values from associated batches:

- Batch count.
- Performance record count.
- Total rounds produced, excluding cancelled batches.
- Average velocity.
- Average standard deviation.

- Average extreme spread.
- Average rating.
- Linked performance records.

The system distinguishes raw records from derived aggregate values.

12.5 Editing and Audit

Performance data may be edited.

When edited:

- `edited` is set to true.
- Previous and new values are audited.
- Created and updated timestamps remain available.

13. Safety, Verification, and Acknowledgement Requirements

13.1 Safety Philosophy

The application prioritizes traceability, consistency, and user acknowledgement.

The application shall not claim that a recipe is safe, recommend load data, or infer safe powder charges.

13.2 Verification Rules

Initial verification includes:

- Required recipe fields are populated.
- Recipe has cartridge/caliber.
- Recipe has bullet, powder, primer, and case components.
- Recipe has at least one source material reference or an audited missing-source acknowledgement.
- Batch quantity is positive.
- Recipe component quantities are positive.
- Inventory lot quantities are sufficient.
- Unit conversions are valid.
- Count-based quantities are whole numbers.
- Batch allocations exactly match required component totals.
- Batch consumption traces to exact lots.
- Container assignments do not exceed batch quantity or container capacity.
- Public recipe view does not expose private inventory data.

13.3 Warnings

The application displays or returns warnings/errors when:

- Required recipe fields are missing.

- Source material is not attached or referenced.
- A recipe is incomplete.
- A batch cannot be produced from selected inventory.
- A selected container already contains another batch.
- Performance data has been modified from original entry.
- A recipe is not approved but is being used for batch creation.
- Inventory adjustment is attempted while inventory is reserved.

13.4 User Acknowledgements

Acknowledgement records include:

- User.
- Timestamp.
- Entity type.
- Entity identifier.
- Acknowledgement type.
- Text/version.
- Related warning, if any.

Implemented acknowledgement types include:

- `RECIPE_RESPONSIBILITY`.
- `MISSING_SOURCE_RECIPE_TRANSITION`.
- `MISSING_SOURCE_BATCH`.
- `NON_APPROVED_RECIPE_BATCH`.
- `MIXED_BATCH_CONTAINER`.
- `INVENTORY_RETURN_LOSS`.

Acknowledgements are audited.

14. User Interface Requirements

14.1 UI Approach

The UI is a server-rendered Flask/Jinja browser interface with static JavaScript enhancements.

It favors traceability, correctness, and clear workflow feedback over visual complexity.

14.2 Main Pages

Implemented pages include:

- Login.
- Registration.
- Account reset workflow.
- Dashboard.
- Items.

- Inventory lots.
- Recipes.
- Public recipe view.
- Batches.
- New batch.
- Batch detail.
- Storage containers.
- Audit/history.
- Settings.
- QR display/download.

14.3 Dashboard

The dashboard shows:

- Item count.
- Current active inventory lot count.
- Depleted inventory lot count.
- Low inventory indicators.
- Recipe count by state.
- Batch count by state.
- Container count by state.
- Batches under production.
- Recent activity.

14.4 Items UI

The user can:

- Create items.
- View items.
- Edit items.
- Archive items.
- Filter by category.
- Search by manufacturer, product line, name, or characteristics.
- Enter category-specific fields.
- Enter flexible JSON attributes.

14.5 Inventory UI

The user can:

- Add inventory lots.
- Select input units.
- Enter manufacturer lot numbers.
- View active inventory.
- Show or hide depleted lots.
- View original quantity and unit.

- View normalized balance.
- View available, reserved, consumed, and adjustment quantities.
- Activate or deactivate a lot according to active-lot rules.
- Replace the active lot during lot creation.
- Create audited inventory adjustments.
- Deplete remaining available inventory.
- View adjustment history.

14.6 Recipe UI

The user can:

- Create a recipe.
- Use a suggested two-word title.
- Select exact item components.
- Enter recipe parameters.
- Attach or link source metadata.
- Enter private and public notes.
- View warnings.
- Acknowledge warnings.
- Change recipe state.
- Mark recipe private or public.
- Generate a public link.
- View aggregate performance.
- View batches produced from the recipe.

The UI does not support recipe component alternatives.

14.7 Batch UI

The user can:

- Create a batch from a recipe.
- Define number of recipe iterations.
- Select exact inventory lots.
- Enter explicit multi-lot allocation JSON.
- See required quantities.
- See inventory availability.
- Transition batch lifecycle state.
- Cancel a batch after explicit return/loss accounting.
- Decommission eligible produced/storage/depleted batches.
- Perform inventory return.
- View inventory reservation and consumption details.
- Enter or edit performance/quality data after production.
- View assigned, unassigned, and depleted container quantities.

14.8 Storage Container UI

The user can:

- Create containers with a cartridge limit.
- Edit containers.
- View current container state.
- Assign batches to containers.
- Track quantity per batch in a container.
- View remaining capacity.
- View mixed-batch warnings.
- Acknowledge mixed-batch assignment.
- Transition containers through `ASSIGNED`, `PARTIALLY USED`, `USED`, and `EMPTY`.
- Generate and download QR labels.

Container retirement is not implemented.

14.9 Performance/Quality UI

The user can:

- Enter one consolidated performance/quality record per batch.
- Enter chronograph-compatible data fields.
- Enter perception metrics.
- Enter notes.
- Edit performance data.
- See whether data has been altered.
- View raw and processed data.
- View recipe-level aggregation.

14.10 Settings UI

The user can:

- Create a SQLite backup.
- Download tenant-scoped exports in JSON or CSV for supported entities.
- Download help/context text.

15. API Requirements

The storage service exposes JSON API endpoints for:

- Health.
- Authentication: register, login, reset, logout, current user.
- Items.
- Inventory lots.
- Inventory adjustments.
- Recipes.

- Recipe suggested identity.
- Recipe components.
- Source material.
- Recipe lifecycle transitions.
- Acknowledgements.
- Public recipe access.
- Batches.
- Batch lifecycle transitions.
- Inventory returns.
- Storage containers.
- Container assignments.
- Performance/quality records.
- Dashboard metrics.
- Audit logs.
- QR code generation.
- Tenant exports.
- SQLite backup.

API errors are structured:

- Machine-readable error code.
- Human-readable message.
- Optional field-level details.

16. Data Integrity Requirements

16.1 General Rules

All tenant-scoped records belong to exactly one user.

All write operations enforce ownership.

All private read operations enforce ownership.

All references between entities are validated before use.

16.2 Soft Deletion and Archiving

Traceability records are not hard-deleted through normal workflows.

The system uses:

- Item archival.
- Recipe archival.
- Batch cancellation.
- Batch depletion.
- Batch decommissioning.
- Inventory lot depletion.
- Container emptying.

Hard deletion is limited to development and test maintenance through the `delete-user` CLI command.

16.3 Audit Logging

The application maintains audit records for important changes, including:

- User creation and login.
- Password reset requirement and reset.
- Item creation and update.
- Inventory lot creation, activation, deactivation, opening, reservation, consumption, and adjustment.
- Recipe creation, update, sharing change, component creation/deletion, source creation, and state change.
- Batch creation and state change.
- Container creation, update, assignment, and assignment clearing.
- Inventory return/loss.
- Performance record creation and update.
- Safety acknowledgements.
- Export creation.
- Backup creation.

Audit records include:

- User.
- Timestamp.
- Entity type.
- Entity identifier.
- Action.
- Previous value where reasonable.
- New value where reasonable.
- Notes or reason where applicable.

17. Database and Migration Requirements

17.1 Database

The current implementation uses SQLite.

The database file is stored in a Docker volume at `/data/reloading.sqlite3` so data persists across container recreation.

17.2 ORM

SQLAlchemy defines persistent entities and relationships.

Flask-SQLAlchemy manages application integration.

17.3 Migrations

Alembic migrations are used and are run on storage container startup.

Current migration history includes:

- `0001_initial`: initial traceability schema.
- `0002_inventory_adjustments`: adds `adjustment_quantity` and `inventory_adjustment`.
- `0003_remove_recipe_alternatives`: nulls `alternative_group` and disables alternatives.
- `0004_recipe_uuid_identifiers`: replaces recipe slugs with UUID identifiers.
- `0005_batch_uuid_identifiers`: adds UUID batch identifiers while retaining friendly slugs.
- `0006_container_cartridge_limit`: adds `cartridge_limit`.
- `0007_revised_batch_states`: renames batch depletion states.
- `0008_reconcile_assigned_batch_states`: repairs historical assigned batches under derived state behavior.
- `0009_batch_container_depleted_quantity`: tracks quantity cleared from emptied containers.

17.4 Backup and Export

The application supports SQLite backup through `/api/admin/backup` and the Settings UI.

Backups are written to `/data/backups`.

The application supports tenant-scoped JSON and CSV exports for:

- Items.
- Inventory.
- Recipes.
- Batches.
- Containers.
- Performance records.
- Audit records.

18. Testing Requirements

18.1 Unit Testing

Pytest is used for unit tests.

Unit coverage includes:

- Powder unit conversion.
- Count unit validation.
- Slug generation and collision handling.

- Transition validation.
- Slug word list capacity.

18.2 Functional API Testing

Functional tests cover:

- Tenant isolation.
- Active lot rule.
- Active lot replacement.
- Category-specific item field filtering.
- Public recipe privacy.
- Recipe component uniqueness by core role.
- Recipe component role derivation from item category.
- Suggested recipe identity generation.
- Recipe source warnings and acknowledgements.
- Batch missing-source acknowledgement.
- Reservation, consumption, and depletion.
- Blocking premature performance entry.
- Container capacity and assignment quantities.
- Automatic batch state updates from containers.
- Container emptying and batch depleted quantity.
- Legacy assigned-batch reconciliation.
- System-managed opened date.
- Inventory adjustments, deplete remaining, restore, and validation.
- Blocking adjustment while reserved.
- Shortage rollback.
- Cancellation with explicit return/loss accounting.

18.3 Browser Workflow Testing

Selenium workflow tests are opt-in.

The current browser test covers a .357 Magnum workflow:

- Registration.
- Failed and successful login.
- Logout and relogin.
- Item creation across categories.
- Inventory lot creation.
- Dashboard metrics.
- Recipe creation and approval.
- Related recipe flows, including UI-level prevention of duplicate core components.
- Public recipe link creation and public-safe view.
- Automatic replacement-lot selection for successor-lot promotion.
- Batch creation and production.
- Performance record entry.

- Garmin FIT performance import.
- Container creation and assignment.
- Capacity overfill rejection.
- Mixed-batch acknowledgement.
- Storage and depletion state checks.
- Audit presence checks.

18.4 Migration Testing

Migration behavior is exercised through container startup and migration scripts. Dedicated migration tests are still a future hardening area.

18.5 Error Handling Tests

Tests verify predictable errors for:

- Invalid units.
- Negative or invalid quantities.
- Insufficient inventory.
- Invalid lifecycle transitions.
- Unauthorized cross-tenant access.
- Missing required recipe fields.
- Premature performance data entry.
- Inventory adjustment while reserved.
- Private data exposure through public recipe links.

19. Current Implementation Phases

Phase 1: Project Foundation

Status: Implemented.

Includes Docker Compose, three-service architecture, Flask storage service, Flask rendering service, Nginx entrypoint, SQLite volume, SQLAlchemy, Alembic, pytest, health checks, and configuration.

Phase 2: Authentication and Multi-Tenant Foundation

Status: Implemented.

Includes user model, registration, login, password hashing, bearer sessions, tenant ownership enforcement, reset-required workflow, dashboard, and authorization tests.

Phase 3: Items and Inventory Lots

Status: Implemented and extended.

Includes item CRUD, categories, flexible attributes, inventory lot creation/update, unit normalization, active lot rule, active replacement, historical lot filtering, opened-date behavior, and inventory dashboard metrics.

Phase 4: Recipes and Source Material

Status: Implemented with changes.

Includes recipe CRUD, UUID identifiers, suggested two-word titles, lifecycle states, exact item components, source metadata, warnings, acknowledgements, public/private state, and public recipe link.

Recipe alternatives were removed/deferred.

Phase 5: Batch Production and Inventory Reservation

Status: Implemented and revised.

Includes batch creation, UUID identifiers, friendly slugs, explicit lot allocation, reservation on creation, consumption on transition to `PRODUCED`, shortage rollback, multi-lot traceability, and batch edit restrictions.

Phase 6: Inventory Return, Cancellation, and Decommission

Status: Partially implemented.

Implemented cancellation accounting, return/loss workflow, return to source or existing compatible lot, loss tracking, and audit history.

Return-to-new-lot inside the return workflow is deferred.

Phase 7: Storage Containers and QR Labels

Status: Implemented and extended.

Includes container CRUD, cartridge limits, capacity enforcement, batch-to-container assignment, split batches, mixed-batch containers, quantity per batch, mixed-batch acknowledgement, QR generation, and downloadable QR images.

Container retirement and full occupancy history are deferred.

Phase 8: Performance and Quality Records

Status: Implemented.

Includes one consolidated record per batch, chronograph-compatible fields, perception metrics, raw and processed data fields, edit workflow, edited indicator, audit trail, and recipe-level aggregation.

Phase 9: Backup, Export, and Hardening

Status: Implemented in core form.

Includes backup workflow, JSON export, CSV export, structured errors, functional tests, Selenium workflow tests, and documentation in README.

Dedicated migration tests and pre-migration automatic backup remain hardening candidates.

20. Non-Goals for Current Implementation

The current implementation shall not include:

- Load-data recommendations.
- Automatic safe-load inference.
- AI-generated recipe suggestions.
- Google OAuth.
- Email-based password recovery.
- Recipe component alternatives.
- Direct source file upload.
- Return-to-new-lot within the return endpoint.
- Nested storage containers.
- Full historical occupancy tracking for containers.
- Mobile-native application.
- Multi-user collaborative editing.
- Public sharing beyond recipes.
- Inventory marketplace or purchasing integration.

21. Future Expansion Candidates

Future versions may add:

- Google OAuth login.
- Email-based account recovery.
- Direct source file upload.
- Recipe component alternatives through explicit recipe variants or controlled alternatives.
- Return-to-new-lot during inventory return.
- More advanced recipe analytics.
- Load-data comparison against user-entered source limits.
- Printable batch sheets.
- Printable container labels.
- Barcode scanning.
- QR scanning from mobile devices.
- Full container history.
- Container retirement.
- Role-based sharing beyond public/private recipe links.

- PostgreSQL support.
- Admin console.
- Mobile-friendly offline mode.
- AI-assisted analysis of accumulated performance data, subject to strict safety boundaries.

22. Acceptance Criteria Summary

The application is currently aligned with these acceptance criteria:

- A user can register, log in, log out, and access only their own data.
- A user can reset a password through an operator-triggered local reset workflow.
- A user can create, edit, list, search, filter, and archive Items.
- A user can add lot-based Inventory.
- Powder inventory can be entered in common mass units and consumed in grains.
- Count inventory requires whole-number count quantities.
- Depleted lots remain historically visible.
- Inventory can be adjusted through audited correction records.
- A user can create a Recipe for one completed cartridge.
- A Recipe can reference exact component Items.
- A Recipe can include source metadata and safety acknowledgements.
- A Recipe can be private or public by public link.
- Public recipe access does not expose private tenant data.
- A user can create a Batch from a Recipe.
- A Batch reserves inventory under production.
- A Batch consumes inventory when moved to `PRODUCED`.
- A Batch cannot override insufficient inventory.
- A Batch can be cancelled only with explicit return/loss accounting.
- A Batch can be assigned across one or more containers.
- A Container can hold one or more batches with acknowledgement.
- Container capacity limits are enforced.
- Batch storage and depletion states update from container assignments and container state.
- QR codes can be generated for batch and recipe identification.
- A Batch can have one consolidated performance/quality record after production.
- Recipe pages can aggregate batch performance data.
- Important changes are audited.
- The system can be migrated through Alembic.
- The system can be backed up.
- Tenant data can be exported as JSON or CSV.
- API and Selenium tests cover major workflows.